

지하철 연착 확률 메트릭

약간 쉬운 비유를 위해: 야구에는 OAA라고, 캐치확률이 존재합니다.

수비수의 **시작 위치**부터 최종 낙구위치까지의 **거리**, 공이 떨어지는데 걸린 **시간**

두 가지를 바탕으로 이 타구를 아웃 시킬(캐치할) 빅데이터적 통계 확률을 나타냅니다.

이와 비슷하게, 지하철의 현 실시간 위치와 내가 타겟으로 하는 역에 이 지하철이

늦게 올 확률/ 빨리올 확률(기준은 1분, 변할 수 있음), 써있는 시간에 올 확률을

실제 이벤트, 30분 지연 분포확률 등을 바탕으로 제공하는 시스템을 구축해보고자 합니다.

RailOdds 프로젝트 기획서

지하철 도착 예측의 신뢰도를 실측 누적으로 계량화하는 실시간 정보 서비스

범위: 서울교통공사 1~9호선 실시간 도착·위치 데이터

MVP 기간: 4주

1. 기획 결론

RailOdds는 "몇 분 후 도착"이라는 **점 추정치** 대신 **확률 분포**를 제공한다.

구분	기존 서비스	RailOdds
출력 형태	"3분 후 도착"	"3분 후 예정 · 정시 48% / 지연 34% / 조기 18%"
근거	관제 시스템 예측값 그대로 전달	예측값 + 실측 누적 잔차 분포
오차 설명	없음	시스템 지연분과 운행 편차분으로 분해
신뢰도	표시하지 않음	관측 건수 기반으로 명시

한 줄 소개

같은 역·같은 시간대에서 그동안 예측이 실제로 얼마나 빗나갔는지를 누적해, 지금 눈앞의 "3분 후 도착"을 얼마나 믿어야 하는지 확률로 알려준다.

핵심 가설

사용자는 이미 도착 예측이 자주 틀린다는 것을 경험적으로 알고 있으며, 매번 감으로 여유 시간을 보정하고 있다. 그 보정을 데이터로 대신해주면 사용자는 출발 시점을 더 정확하게 결정할 수 있다.

RailOdds의 확률은 열차 운행을 더 정확히 예측하겠다는 주장이 아니라, **기존 예측값의 신뢰 구간을 제시하는 보조 지표**다.

지표 설계 — AOE

$$AOE = \sum (\text{실제 도착 시각} - \text{예측 도착 시각})$$

역·방향·시간대 단위로 누적하며, 표본이 적을 때 값이 튀는 것과 관측이 쌓이며 안정화되는 것 모두 OAA와 동일한 성질을 갖는다. 초기 불완전성을 결함이 아니라 설계된 특성으로 명시한다.

2. 해결하려는 문제

- "3분 후 도착"이 실제로는 1분일 때도, 6분일 때도 있으며 사용자는 그 편차 폭을 알 수 없다.
- 오차의 원인이 데이터 전달 지연인지 실제 운행 편차인지 구분되지 않는다.
- 모든 앱이 동일한 원천(TOPIS) 예측값을 그대로 전달할 뿐, 그 값의 과거 적중률을 추적하지 않는다.
- 역과 시간대에 따라 오차 특성이 크게 다른데도 단일한 형태로 표시된다.
- 사용자는 결과적으로 "대충 더 여유 두고 나가기"라는 개인적 보정에 의존한다.

3. 주요 사용자와 제공 결과

사용자	목적	서비스가 제공하는 결과
출퇴근 이용자	언제 집을 나설지 결정	자주 쓰는 역·시간대의 정시 확률과 권장 여유 시간
환승 이용자	환승 성공 여부 판단	현재 열차 기준 다음 열차 연결 확률
일반 이용자	지금 뛰어야 하는지 판단	도착 예측의 신뢰 구간
운영·분석 관점	예측 품질 진단	역별 AOE 랭킹, 오차 원인 분해 리포트

마지막 항목은 실사용자용은 아니지만 **데모에서 가장 강력한 산출물**이다. "서울에서 도착 예측이 가장 안 맞는 역 TOP 10"과 "그중 몇 초가 시스템 지연이고 몇 초가 운행 편차인가"는 다른 어떤 서비스도 제시하지 않는 결과다.

4. 사용자 흐름

1. **역 선택** — 자주 쓰는 역과 방향을 등록한다.
2. **도착 조회** — 기존 앱과 동일하게 예측 도착 시각을 본다.
3. **신뢰도 확인** — 예측값 아래에 정시·지연·조기 확률과 관측 건수가 함께 표시된다.
4. **여유 시간 권고** — "정시 확률 90%를 원하면 2분 일찍 나가세요" 형태로 행동 지침을 제공한다.
5. **누적 리포트** — 이 역의 AOE 추이와 시간대별 오차 패턴을 확인한다.
6. **신뢰도 성장 확인** — 관측이 쌓이며 추정이 정교해지는 과정이 화면에 드러난다.

5. 핵심 기능과 화면

화면	핵심 기능	MVP 완료 기준
역 조회	도착 예측 + 확률 분포 표시	3구간 확률과 관측 건수 동시 노출
상세 신뢰도	잔차 히스토그램, 리드타임별 정확도	예측 시점별로 정확도가 다름을 시각화
여유 시간 계산	목표 정시 확률 입력 시 권장 출발 시각	사용자가 임계값을 직접 조정 가능
AOE 랭킹	역·노선별 누적 편차 순위	시스템 지연분과 운행 편차분 분리 표시
오차 분해 리포트	recptnDt 기반 파이프라인 지연 분석	두 성분의 비율을 정량 제시
운영 대시보드	처리량, 컨슈머 랙, 체크포인트 상태	장애 주입 시연 시 실시간 반영

6. 데이터 설계

6.1 입력 소스

소스	갱신	용도	비용
실시간 도착정보(일괄)	10초 폴링	예측값, 열차번호, 도착 상태	무료
실시간 열차 위치정보	30초 폴링	실제 위치 검증, 급행 구분	무료
서울교통공사 30분 혼잡도	분기 갱신	시간대별 평상시 혼잡 프로파일	무료
역 마스터(xlsx)	비정기	역명 표기 정규화	무료

역 단위 개별 조회가 아닌 **일괄 API를 기준으로 설계**한다. 300개 역을 개별 폴링하면 호출량이 감당되지 않고 원천 서버에도 부담이 된다.

6.2 수집 원칙

- **원본 응답을 가공 없이 적재한다.** 수집 시점에 스키마를 확정하면 되돌릴 수 없다. 파싱은 사후에 반복 가능하다.
- **수신 시각을 반드시 기록한다.** `recptnDt` (데이터 생성 시각)와 수신 시각의 차이가 파이프라인 지연이며, 이 프로젝트의 핵심 분석 대상이다.
- **수집기는 죽지 않는다.** 어떤 예외에도 다음 폴링으로 넘어가며 며칠간 무인 운영된다.
- 과거 데이터가 제공되지 않으므로 **직접 수집한 것만이 데이터셋**이다. 개발 착수와 무관하게 수집을 선행한다.

6.3 표준 이벤트 스키마

```
event_id, station_id, line_id, direction, train_no,
predicted_arrival_sec, arrival_status_code,
recptn_dt, received_at, lead_time_sec, quality_flag
```

잔차 레코드는 별도 스키마로 적재한다.

```
station_id, direction, hour_bucket, dow, train_no,
lead_time_sec, predicted_sec, actual_sec,
residual_sec, pipeline_lag_sec, operation_dev_sec,
congestion_ref, headway_sec, model_version
```

`residual_sec = pipeline_lag_sec + operation_dev_sec` 로 분해 가능하도록 두 성분을 각각 저장한다.

6.4 파생 특징

입력	파생 특징	용도
예측 스냅샷 시계열	리드타임별 예측값, 수렴 안정성	예측 시점에 따른 정확도 차이
recptnDt vs 수신 시각	파이프라인 지연	오차 원인 분해
도착 상태 코드 전이	실제 도착 시점	잔차 계산의 정답값
선형 열차 도착 이력	배차 간격(headway)	동적 혼잡 프록시
30분 혼잡도 테이블	평상시 혼잡 수준	정적 혼잡 기준선

혼잡도는 **정적 층과 동적 층으로 분리**한다. 정적 층은 broadcast state로 배포되는 참조 데이터이고, 동적 층은 도착 API만으로 계산되는 실시간 배차 간격이다. 공사 혼잡도가 반영하지 못한다고 명시한 운행 간격과 특이 이벤트를 동적 층이 보완한다.

혼잡도는 결과 변수이기도 하므로(지연 → 승객 누적 → 정차 지연) 피처로 사용할 때 **예측 시점 이전의 값만 참조**한다.

7. 예측 로직

7.1 1단계: 예측 스냅샷 누적

동일 열차에 대해 리드타임이 다른 예측 스냅샷이 반복 수신된다. Flink는 열차번호를 키로 이 스냅샷들을 상태에 보관한다.

7.2 2단계: 실제 도착 매칭

도착 상태 코드가 도착으로 전이되는 시점을 실제 도착으로 확정하고, 보관 중이던 모든 스냅샷과 짝지어 잔차를 계산한다. **한 번의 도착이 여러 개의 학습 샘플을 생성**하며, 리드타임 자체가 중요한 피처가 된다. 10분 전 예측의 오차와 1분 전 예측의 오차는 다른 문제다.

매칭 완료 후 열차 상태는 즉시 정리하고, 누락에 대비해 TTL을 설정한다.

7.3 3단계: 조건부 분포 추정

역·방향·시간대를 키로 잔차 분포를 유지한다. 원본 잔차를 모두 보관하면 상태가 무한히 증가하므로 **t-digest로 압축**하고 관측 건수를 함께 저장한다.

7.4 4단계: 계층적 사전분포

관측이 부족한 조합은 상위 레벨 분포를 사전분포로 사용하고, 관측이 쌓이면 개별 분포로 이행한다.

$$\begin{aligned}\text{추정 분포} &= w(n) \times \text{개별 분포} + (1 - w(n)) \times \text{상위 분포} \\ w(n) &= n / (n + k)\end{aligned}$$

상위 레벨은 노선×시간대, 그다음 노선 전체 순으로 폴백한다. 가짜 데이터로 빈 칸을 채우지 않으며, 모든 값은 실제 관측의 집계다. 사용자에게는 "관측 23건, 노선 평균 기반 추정"과 같이 근거를 그대로 노출한다.

7.5 출력 형식

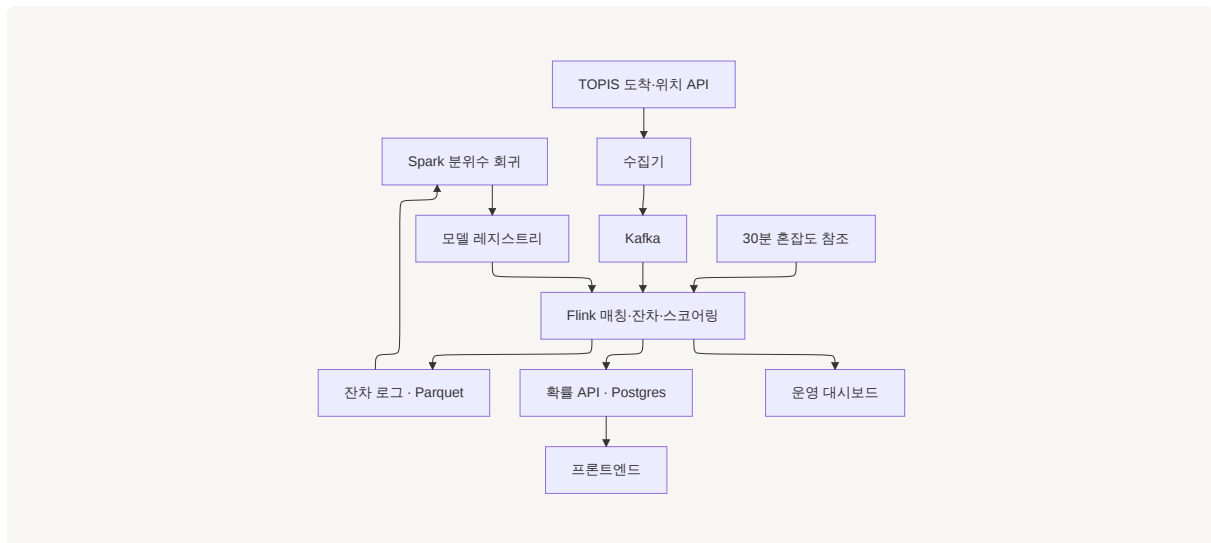
3분 후 도착 · 정시 확률 48%

1분 이상 지연 34%
±1분 내 정시 48%
1분 이상 조기 18%

신뢰도: 관측 1,240건 · 이 역 고유 분포 기준
권장: 정시 확률 90%를 원하면 2분 일찍 출발

"AI가 정확히 예측", "오차 0" 같은 표현은 사용하지 않는다. 원천 예측값을 대체하지 않고 그 위에 신뢰 구간을 덧붙이는 위치를 유지한다.

8. 시스템 아키텍처



컴포넌트 책임

컴포넌트	책임
수집기	일괄 폴링, 원본 적재, 수신 시각 기록, 무중단 운영
Kafka	이벤트 버퍼, 역x방향 파티션 키, 리플레이 소스
Flink	예측 스냅샷 상태 관리, 도착 매칭, 잔차 계산, 분포 갱신, 확률 스코어링
MinIO/HDFS	잔차 로그 Parquet 적재
Spark	분위수 회귀 학습, 캘리브레이션 평가, ONNX 내보내기
Postgres	최신 확률 스냅샷, AOE 누적, 서빙용 조회
프론트엔드	역 조회, 신뢰도 표시, AOE 랭킹
대시보드	처리량, 랙, 체크포인트, 장애 복구 시연

처리 순서

1. 수집기가 일괄 API를 폴링해 원본과 수신 시각을 Kafka에 발행한다.
2. Flink가 열차번호 키로 예측 스냅샷을 상태에 보관한다.
3. 도착 상태 전이를 감지하면 보관 스냅샷 전체와 매칭해 잔차를 산출한다.

4. 잔차를 파이프라인 지연분과 운행 편차분으로 분해해 분포에 반영한다.
5. 스코어링 시점의 피쳐 벡터를 그대로 Parquet에 적재한다.
6. Spark가 원본 API 응답이 아닌 이 잔차 로그로 학습한다.
7. 모델 버전이 Kafka로 발행되고 Flink가 broadcast state로 무중단 교체한다.

5번과 6번이 이 아키텍처의 핵심이다. 학습 피쳐가 서빙 코드 경로에서 생성되므로 **학습·서빙 피쳐 불일치**가 구조적으로 발생할 수 없다.

9. 핵심 데이터 모델

테이블	핵심 필드
<code>stations</code>	역 ID, 노선, 방향, 표기 정규화 매핑
<code>raw_snapshots</code>	원본 응답, recptnDt, 수신 시각, 폴링 회차
<code>residuals</code>	역·방향·시간대, 리드타임, 잔차, 지연분, 편차분
<code>distributions</code>	키, t-digest 직렬화, 관측 건수, 갱신 시각
<code>aoe_cumulative</code>	역·방향, 누적 AOE, 표본 수, 기간
<code>congestion_ref</code>	요일, 호선, 역, 상하선, 30분 단위 혼잡도
<code>model_versions</code>	버전, 학습 기간, 캘리브레이션 지표, 배포 시각
<code>serving_snapshots</code>	최신 확률, 신뢰도 등급, 사용 모델 버전

모든 확률 출력에는 `model_version` 과 관측 건수를 저장해 재현과 사후 비교가 가능해야 한다.

10. 대규모 데이터와 분산 처리의 역할

수집 초기의 실제 데이터량만으로는 분산 처리의 필요성을 증명하기 어렵다. 따라서 용도를 분리한다.

데이터	용도	주의점
실제 수집 데이터	잔차 분포 추정, 서비스 동작	기간이 짧아 콜드스타트 존재
배속 리플레이	처리량·레이턴시 벤치마크	부하 생성용이며 학습에 사용하지 않음
증폭 리플레이	Flink 한계 탐색, 백프레셔 관찰	합성 부하임을 명시

리플레이는 부하 생성기이지 학습 데이터가 아니다. 이 구분을 흐리면 확률값의 근거가 사라진다.

성능시험은 1배속·5배속·20배속 리플레이에서 처리량, p50/p99 레이턴시, 컨슈머 랙을 측정하고, TaskManager와 Kafka 브로커를 각각 강제 종료해 복구 시간과 중복·유실 건수를

기록한다.

11. 기술 스택과 배포

영역	기술
수집	Python, requests
메시징	Apache Kafka
스트림 처리	Apache Flink
배치 학습	Apache Spark, LightGBM 분위수 회귀
모델 포맷	ONNX (Flink 오퍼레이터 내장)
저장	MinIO(S3 호환), Parquet, Postgres
시각화	Grafana, 프론트엔드 프레임워크
배포	Docker Compose, Linux

의도적으로 제외한 것: Kubernetes, Airflow, Hadoop/YARN, 딥러닝, 별도 모델 서빙 REST 서버. 4주 일정에서 필요하지 않은 인프라는 부채다. 특히 모델 서빙을 분리하면 트랜잭션당 네트워크 홉이 추가되어 레이턴시 측정이 오염된다.

12. 4주 개발 계획

주차	핵심 작업	완료 기준
0주차	수집기 선행 가동	API 응답 구조 확인, 무종단 수집 시작
1주차	docker-compose 구성, Kafka 발행, 카운트만 하는 Flink 잡	체크포인트 활성화 상태로 end-to-end 흐름
2주차	예측 스냅샷 상태, 도착 매칭, 잔차 계산, Parquet 적재	잔차 로그 생성 및 Spark 학습 아티팩트
3주차	ONNX 스코어링, 모델 핫 리로드, 확률 API, 화면	라이브 데모 동작
4주차	장애 주입, 벤치마크, AOE 리포트, 발표 준비	재현 가능한 실험표와 최종 문서

0주차 수집은 다른 작업과 병렬로 진행한다. 데이터는 시간이 지나야 축적되므로 지연된 만큼 회복이 불가능하다.

1주차 골격이 안정되기 전에는 피쳐 작업을 시작하지 않는다. 이후 모든 작업이 이 골격 위에 쌓인다.

13. 팀 분담

역할 정의

역할	담당 범위	주요 산출물
인프라·수집	docker-compose, Kafka 구성, 수집기 운영, 장애 주입 실험	무중단 수집 로그, 복구 시간 측정표
스트림 처리	Flink 잡, 상태 설계, 매칭 로직, TTL, broadcast state	잔차 생성 파이프라인, 상태 크기 추이
모델·분석	Spark 학습, 분위수 회귀, 계층적 사전분포, 캘리브레이션	신뢰도 다이어그램, 오차 분해 리포트
서비스	확률 API, 프론트엔드, AOE 랭킹 화면, 대시보드	데모 화면, 운영 대시보드

인원별 편성안

4인 구성

인원	담당
A	인프라·수집 + 장애 실험
B	스트림 처리 (Flink 전담)
C	모델·분석 (Spark, 캘리브레이션)
D	서비스 (API + 프론트)

5인 구성

인원	담당
A	인프라·수집
B, C	스트림 처리 — B는 매칭·상태, C는 스코어링·핫 리로드
D	모델·분석
E	서비스

6인 구성

인원	담당
A	인프라·Kafka
B	수집기·데이터 품질·장애 실험
C, D	스트림 처리
E	모델·분석
F	서비스·시각화

분담 원칙

- **Flink 담당은 반드시 2명 이상 관여한다.** 스트림 상태 디버깅은 배치 대비 3배 이상 느리고, 단일 담당자가 막히면 전체 일정이 정지한다. 6인 구성에서 Flink에 2명을 배치하는 이유다.
- **3주차 말까지 세로 흐름을 먼저 완성한다.** 수집 → 매칭 → 학습 → 스코어링 → 화면의 한 줄이 얇게라도 관통해야 한다. 각 영역을 병렬로 깊게 파면 통합 시점에 무너진다.
- **4주차는 신규 기능을 넣지 않는다.** 장애 실험과 측정, 문서화에만 사용한다. 이 주차가 평가 대상 산출물을 만든다.
- **인프라 담당이 1주차 병목이다.** 1주차에는 다른 인원이 스키마 설계, 화면 와이어프레임, 학습 코드 골격 등 인프라 비의존 작업을 선행한다.
- 담당자별로 **작은 리플레이 파일(1천 행)** 을 공유해 빠른 반복이 가능하게 한다. 전체 데이터로 매번 검증하면 개발 속도가 급락한다.

14. MVP 범위

포함

- 서울 1~9호선 일괄 도착 데이터 수집 및 무중단 운영
- 예측 스냅샷과 실제 도착 매칭, 리드타임별 잔차 생성
- 오차의 파이프라인 지연분·운행 편차분 분해
- 역·방향·시간대 조건부 확률 분포 추정
- 계층적 사전분포 기반 콜드스타트 대응
- 관측 건수 기반 신뢰도 노출
- AOE 누적 랭킹 및 오차 분해 리포트
- 모델 핫 리로드, 장애 복구, 처리량 벤치마크

제외

- 서울시 외 구간, 광역철도, 신분당선 등 미제공 노선
- 유료 혼잡도 API 연동
- 실시간 칸별 혼잡도
- 경로 탐색, 환승 안내, 지도 서비스
- 푸시 알림, 계정 시스템, 모바일 앱 출시

- 열차 운행 자체를 예측하는 모델(원천 예측값을 대체하지 않는다)
- 지연 원인의 인과 규명(고장, 사고 등 사유 판별)

15. 평가 지표

평가축	MVP 지표
데이터 파이프라인	수집 성공률, 폴링 결측률, 매칭 성공률
스트림 처리	처리량(events/sec), p50·p99 레이턴시, 컨슈머 랙
내결함성	노드 강제 종료 후 복구 시간, 중복·유실 건수 0
상태 관리	체크포인트 소요 시간, 상태 크기 증가 추이
확률 품질	캘리브레이션 — 정시 확률 70%라 한 사례가 실제 70% 정시였는가
분포 추정	피너볼 손실, 구간 적중률
오차 분해	파이프라인 지연분과 운행 편차분의 비율 및 안정성
성장성	관측 누적에 따른 캘리브레이션 개선 곡선

정확도(MAE)를 주 지표로 삼지 않는다. 이 서비스의 목적은 예측을 정확하게 만드는 것이 아니라 **불확실성을 정직하게 표현하는 것**이므로, 캘리브레이션이 핵심 지표다.

16. 경쟁 서비스와 포지셔닝

서비스	방식	RailOdds의 차이
네이버·카카오 지도	TOPIS 예측값 그대로 전달	예측값의 과거 적중률을 누적해 신뢰 구간 제시
지하철 전용 앱	도착 시각 + 혼잡도 표시	오차 원인을 시스템 지연과 운행 편차로 분해
TOPIS 공식	원천 관제 정보 제공	원천을 대체하지 않고 그 위에 신뢰도 층을 추가
유료 혼잡도 API	실시간 칸별 혼잡도	무료 소스 조합으로 배차 간격을 동적 프록시로 대체

방어 가능한 문구는 다음과 같다.

동일한 공개 예측값을 쓰면서도, 그 예측이 그동안 얼마나 맞았는지를 실측으로 누적해 신뢰 구간과 오차 원인을 함께 제시하는 도착 정보 서비스

"가장 정확한 지하철 앱"은 주장하지 않는다. 원천 예측값 자체를 개선하는 것이 아니기 때문이다.

17. 데이터 이용 및 운영 원칙

- 공공누리 1유형(출처표시)이므로 상업적 이용과 변경이 가능하나 **출처를 명시**한다.
- 폴링 주기는 원천 서버에 부담을 주지 않는 수준으로 유지하고, 노선별 호출 사이에 간격을 둔다.
- 개인정보를 일절 수집하지 않는다. 열차 운행 데이터만 다루므로 개인정보 이슈가 발생하지 않는다.
- 수집 실패와 결측을 은폐하지 않고 품질 지표로 함께 공개한다.
- 확률 표시에 관측 건수를 항상 병기해 근거 없는 확신을 주지 않는다.

18. 주요 위험과 대응

위험	대응
도착 상태 코드로 실제 도착을 잡지 못함	착수 전 최우선 검증 항목. 불가 시 예측 소멸 시점으로 근사하되 정확도 저하를 명시
Flink 상태 무한 증가	열차 상태는 매칭 후 즉시 정리, TTL 초기 설정 필수
수집 기간 부족으로 표본 희소	계층적 사전분포, 신뢰도 명시, 특정 역·시간대에 집중
API 엔드포인트·스키마 변경	원본 그대로 적재해 사후 재파싱 가능하게 유지
스트림 디버깅 지연	소형 리플레이 파일 상시 유지, Flink 담당 복수 배치
원천 API 장애	수집기 무중단 설계, 결측 구간을 품질 지표로 기록
혼잡도 실시간성 부재	정적 참조 + 동적 배차 간격 조합으로 대체
4주차 일정 잠식	3주차 말 기능 동결, 4주차는 측정·문서화 전용

19. 기획 단점

단점	왜 문제인지
원천 예측을 개선하지 않음	열차 운행 자체를 더 정확히 예측하는 것이 아니라 기존 예측의 오차를 계량화할 뿐이어서, "결국 늦는 건 똑같지 않나"는 반론이 가능함
수집 기간이 절대적으로 부족	역×방향×시간대 조합이 수천 개인데 4주 수집으로는 각 칸에 관측이 수십건 수준. 계층적 보정으로 완화할 뿐 근본 해결은 아님
실시간 혼잡도 부재	무료 소스로는 지금 이 순간의 혼잡을 알 수 없음. 배차 간격은 프록시일 뿐이며 실제 승객 수와 괴리가 있을 수 있음
정답 검증의 간접성	실제 도착 시점을 API 상태 코드로 추정하므로, 원천 데이터 자체의 오차가 잔차에 섞여 들어감
사용자 확보 경로 부재	기존 지도 앱이 이미 장악한 영역이라 독립 서비스로서의 사용자 유입 경로가 불분명함. 기능 제안이나 API 형태가 현실적
서울 한정	서울시 외 구간은 원천에서 제공하지 않아 확장이 데이터 제약에 묶임

단점	왜 문제인지
오차 분해의 검증 불가	파이프라인 지연분은 recptnDt로 측정 가능하나, 나머지를 모두 운행 편차로 귀속시키는 것은 가정임

20. 최종 데모 시나리오

1. 수집기가 며칠간 무중단으로 축적한 데이터 규모를 제시한다.
2. 특정 역의 예측 스냅샷이 실제 도착으로 수렴하는 과정을 시각화한다.
3. 같은 역·다른 시간대의 잔차 분포가 뚜렷이 차이를 보여준다.
4. 관측 건수가 적은 역과 많은 역의 신뢰도 표시 차이를 비교한다.
5. 오차를 파이프라인 지연분과 운행 편차분으로 분해한 리포트를 제시한다.
6. AOE 랭킹으로 예측이 가장 안 맞는 역을 공개한다.
7. 캘리브레이션 다이어그램으로 확률의 정직성을 입증한다.
8. 20배속 리플레이 중 TaskManager를 강제 종료하고, 복구 후 중복·유실이 0임을 보여준다.
9. 모델을 무중단 교체하고 확률 출력이 바뀌는 것을 실시간으로 확인한다.

발표용 한 문장

RailOdds는 "몇 분 후 도착하는가"를 더 정확히 맞히려는 서비스가 아니라, 그 숫자를 얼마나 믿어야 하는지를 실측 누적으로 알려주는 서비스다.